

☀️ Class Test 1 - Computer Graphics and Geometric Modeling (2026) ☀️

💎 Complete Detailed Solutions for Set-3 💎

📌 QUESTION 1

📌 Part (a)

Given the center point coordinate (10, -5) and a radius of 8 units, generate all the points (in the 4th octant) by using both the Bresenham and Midpoint circle drawing algorithms. Also, compare the efficiency of both algorithms in terms of the number of calculations and the computational complexity involved in plotting the points. [Marks: 8]

💡 Mathematical Formulations and Steps

Before applying the final translation to the screen center point $(X_c, Y_c) = (10, -5)$, we first calculate the baseline pixel positions centered at the origin $(0, 0)$ for the standard baseline octant (where x ranges from 0 up to y , meaning $x \leq y$).

To transform these baseline coordinates into the true **4th octant** (where x is negative, y is positive, and $|x| \geq y$, going from the line $x = -y$ down to the negative x -axis), the standard octant symmetry mapping is:

- Origin Point mapped to 4th Octant: $(-y, x)$

Finally, we apply translation to shift the points to the given center:

- $X_{\text{screen}} = -y + X_c = -y + 10$
- $Y_{\text{screen}} = x + Y_c = x - 5$

1 Midpoint Circle Drawing Algorithm

- **Initial Values:** $x = 0, y = 8$
- **Initial Decision Parameter:** $p_0 = 1 - R = 1 - 8 = -7$
- **Parameter Update Rules:**
 - If $p_k < 0$: $p(k+1) = p_k + 2x + 3$. Next point is $(x + 1, y)$.
 - If $p_k \geq 0$: $p(k+1) = p_k + 2x - 2y + 5$. Next point is $(x + 1, y - 1)$.

📊 Tracing Table (Midpoint Method)

Step k	Decision Parameter p_k	Point at (0,0)	4th Octant Mapped $(-y, x)$	Final Screen Coordinate $(-y+10, x-5)$
0	Initial	(0, 8)	(-8, 0)	(2, -5)

Step k	Decision Parameter pk	Point at (0,0)	4th Octant Mapped (-y, x)	Final Screen Coordinate (-y+10, x-5)
1	-7	(1, 8)	(-8, 1)	(2, -4)
2	-4	(2, 8)	(-8, 2)	(2, -3)
3	1	(3, 7)	(-7, 3)	(3, -2)
4	-6	(4, 7)	(-7, 4)	(3, -1)
5	3	(5, 6)	(-6, 5)	(4, 0)

(Stop here because the next step would make $x > y$)

2 Bresenham's Circle Drawing Algorithm

- **Initial Values:** $x = 0, y = 8$
- **Initial Decision Parameter:** $d_0 = 3 - 2R = 3 - 16 = -13$
- **Parameter Update Rules:**
 - If $d_k < 0$: $d(k+1) = d_k + 4x + 6$. Next point is $(x + 1, y)$.
 - If $d_k \geq 0$: $d(k+1) = d_k + 4(x - y) + 10$. Next point is $(x + 1, y - 1)$.

Tracing Table (Bresenham Method)

Step k	Decision Parameter dk	Point at (0,0)	4th Octant Mapped (-y, x)	Final Screen Coordinate (-y+10, x-5)
0	Initial	(0, 8)	(-8, 0)	(2, -5)
1	-13	(1, 8)	(-8, 1)	(2, -4)
2	-7	(2, 8)	(-8, 2)	(2, -3)
3	3	(3, 7)	(-7, 3)	(3, -2)
4	-11	(4, 7)	(-7, 4)	(3, -1)
5	7	(5, 6)	(-6, 5)	(4, 0)

Efficiency and Complexity Analysis

1. **Computational Complexity:** Both algorithms require $O(N)$ time complexity, where N is the number of pixels plotted along the circle arc segment. They completely replace expensive floating-point or trigonometric calculations with fast integer additions, subtractions, and bit shifts.
2. **Computational Counts:** The Midpoint method increments the decision parameter using linear variables ($2x + 3$ or $2x - 2y + 5$), while Bresenham's uses structural steps ($4x + 6$ or $4(x - y) + 10$). They make identical pixel-plotting choices because Bresenham's parameter is mathematically related to the Midpoint parameter by $d = 2p - 1$. Therefore, operational speed is identical on modern systems.

✦ Part (b)

Consider a non-interlaced raster monitor with a resolution of n by m (m scan lines and n pixels per scan line), a refresh rate of r frames per second, a horizontal retrace time of $t_{\text{horizontal}}$ and a vertical retrace time of t_{vertical} . What is the fraction of the total refresh time per frame spent in retrace of the electron beam. [Marks: 3]

💡 Solution:

1. **Total Time for One Complete Frame (T_{frame}):** Given a refresh rate of r frames per second, the total allocation for one frame is:

- $T_{\text{frame}} = 1 / r$ seconds

2. **Total Retrace Time Inside One Frame (T_{retrace}):** An electron beam performs a horizontal retrace after every single scan line, and a vertical retrace once at the end of the full screen:

- Total Horizontal Retrace Time = $m * t_{\text{horizontal}}$
- Total Vertical Retrace Time = t_{vertical}
- $T_{\text{retrace}} = (m * t_{\text{horizontal}}) + t_{\text{vertical}}$

3. **Fraction of Total Refresh Time Spent in Retrace:**

- Fraction = $T_{\text{retrace}} / T_{\text{frame}}$
- Fraction = $((m * t_{\text{horizontal}}) + t_{\text{vertical}}) / (1 / r)$
- **Fraction = $r * ((m * t_{\text{horizontal}}) + t_{\text{vertical}})$**

📌 QUESTION 2

✦ Part (a)

A cubic splined curve has start and end point at $(20, 0)$ and $(7, 2)$ respectively. The tangent vectors at start and end points meet at $(18, 9)$. Determine the Hermite cubic spline curve and hence find out its mid-point. [Marks: 7]

💡 Solution:

Given boundary parameters:

- Start position: $P_0 = (20, 0)$
- End position: $P_1 = (7, 2)$
- Tangent intersection vertex: $V = (18, 9)$

1. **Deriving Tangent Vectors:** The tangent vector at the start points from P_0 towards the intersection point V , and the ending tangent points from V towards P_1 :

- $P'(0) = V - P_0 = (18 - 20, 9 - 0) = (-2, 9)$
- $P'(1) = P_1 - V = (7 - 18, 2 - 9) = (-11, -7)$

2. **Hermite Curve Parametric Blending Equation:** $P(t) = (2t^3 - 3t^2 + 1)P_0 + (-2t^3 + 3t^2)P_1 + (t^3 - 2t^2 + t)P'(0) + (t^3 - t^2)P'(1)$

3. **Calculating the Mid-point (t = 0.5):** Substitute $t = 0.5$ into the blending function weight coefficients:

- Weight 1 = $2*(0.125) - 3*(0.25) + 1 = 0.5$
- Weight 2 = $-2*(0.125) + 3*(0.25) = 0.5$
- Weight 3 = $0.125 - 2*(0.25) + 0.5 = 0.125$
- Weight 4 = $0.125 - 0.25 = -0.125$

Now compute coordinate values at $t = 0.5$:

- $X(0.5) = 0.5*(20) + 0.5*(7) + 0.125*(-2) - 0.125*(-11)$
- $X(0.5) = 10 + 3.5 - 0.25 + 1.375 = \mathbf{14.625}$
- $Y(0.5) = 0.5*(0) + 0.5*(2) + 0.125*(9) - 0.125*(-7)$
- $Y(0.5) = 0 + 1 + 1.125 + 0.875 = \mathbf{3.0}$
- **Curve Mid-point: (14.625, 3.0)**

Part (b)

Define the knot values for $n=3$ and $k=3$ where symbols carry their usual meaning. [Marks: 3]

 **Solution:**

In B-spline math context, $n + 1$ represents the control points count (here, $n = 3$ implies 4 control points), and k is the curve order. The total number of knots required in the vector is given by the formula:

- Total Knots = $m + 1 = n + k + 2 = 3 + 3 + 2 = \mathbf{8 \text{ knots}}$

Depending on the configuration requirements, there are two standard ways to define these uniform knot values:

1. **Clamped / Open Uniform Knot Vector (Multiplicity $k=3$ at boundaries):**
 - Knot values: $\{0, 0, 0, 1, 2, 2, 2, 2\}$ or scaled uniformly as $\{0, 0, 0, 1, 2, 3, 3, 3\}$
2. **Periodic Uniform Knot Vector (Equal structural spacing):**
 - Knot values: $\{0, 1, 2, 3, 4, 5, 6, 7\}$

QUESTION 3

The lower left-hand and upper right-hand corners of a rectangular clipping window are at (50, 10) and (80, 40), respectively. Using Sutherland-Cohen algorithm, check and find the visibility portion of the quadrilateral PQRS for P(35,10) Q(65,40), R(65,20), and S(95,10) in the clipping window. [Marks: 9]

 **Solution:**

1 Region Boundary Codes

Outcodes are assigned using 4 bits: **[Top, Bottom, Right, Left]**. Window Limits: $X_{min} = 50$, $X_{max} = 80$, $Y_{min} = 10$, $Y_{max} = 40$

- **Point P(35, 10):** $X < X_{min} \rightarrow$ Outcode is **0001** (Left)
 - **Point Q(65, 40):** Inside all boundaries $\rightarrow$ Outcode is **0000** (Inside)
 - **Point R(65, 20):** Inside all boundaries $\rightarrow$ Outcode is **0000** (Inside)
 - **Point S(95, 10):** $X > X_{max} \rightarrow$ Outcode is **0010** (Right)
-

2 Segment-by-Segment Clipping Execution

● Segment PQ (P(35, 10) to Q(65, 40))

- Codes: $P = 0001$, $Q = 0000$. Logical AND = 0000 (Partially visible candidate).
- Clip P against Left boundary ($X = 50$):
 - Slope $m = (40 - 10) / (65 - 35) = 30 / 30 = 1.0$
 - $Y_{intercept} = 10 + 1.0 * (50 - 35) = 25$
 - New point $P' = (50, 25)$ [Code 0000]
- **Visible Line Component: $P'(50, 25)$ to $Q(65, 40)$.**

● Segment QR (Q(65, 40) to R(65, 20))

- Codes: $Q = 0000$, $R = 0000$.
- Both endpoints are 0000 $\rightarrow$ Trivial Accept.
- **Visible Line Component: $Q(65, 40)$ to $R(65, 20)$.**

● Segment RS (R(65, 20) to S(95, 10))

- Codes: $R = 0000$, $S = 0010$. Logical AND = 0000.
- Clip S against Right boundary ($X = 80$):
 - Slope $m = (10 - 20) / (95 - 65) = -10 / 30 = -0.333$
 - $Y_{intercept} = 20 + (-0.333) * (80 - 65) = 20 - 5 = 15$
 - New point $S' = (80, 15)$ [Code 0000]
- **Visible Line Component: $R(65, 20)$ to $S'(80, 15)$.**

● Segment SP (S(95, 10) to P(35, 10))

- Codes: $S = 0010$, $P = 0001$. Logical AND = 0000.
 - Clip S against Right boundary ($X = 80$):
 - Y is constant at 10.
 - New point $S'' = (80, 10)$ [Code 0000]
 - Clip P against Left boundary ($X = 50$):
 - Y is constant at 10.
 - New point $P'' = (50, 10)$ [Code 0000]
 - **Visible Line Component: $S''(80, 10)$ to $P''(50, 10)$.**
-

Final Summary of Visible Quadrilateral

The clipped visible portion forms a custom polygon with the following sequential vertices:

1. **P'(50, 25)**
2. **Q(65, 40)**
3. **R(65, 20)**
4. **S'(80, 15)**
5. **S''(80, 10)**
6. **P''(50, 10)**